VIRAGRAM

"Your Campus. Your Community."

PROJECT REPORT

For

CSA 5005 - Object Oriented Technology Lab

Submitted By

Name	Roll No.
Vishal Velip	2663
Razak Nadaf	2638

Under the guidance of

Mr. Hanumant Redkar
Mr. Sarvesh Velip

Department of Computer Science & Technology
Goa Business School, Goa University
Taleigao Plateau, Goa

Academic Year 2026–27

ACKNOWLEDGEMENT

We would like to express our sincere gratitude to Goa University and the Department of Computer Science & Technology for providing the academic environment and opportunity to develop this project. The project gave us practical exposure to Android application development, cloud services, database integration, object-oriented programming and software testing.

We are thankful to the faculty members who provided guidance during the academic work and helped us understand how a software project should be planned, implemented and documented. Their suggestions encouraged us to move from an initial campus-social idea to a working application with authentication, social features, campus discovery and administration.

We also acknowledge the role of the tools and platforms used during development, including Android Studio, Java, Firebase, Cloud Firestore, ImageKit, Glide, Node.js, Express, GitHub and Render. Working with these technologies helped us understand how a mobile application can communicate with cloud services and a backend service.

Finally, we thank everyone who contributed directly or indirectly to the development and testing of Viragram. The debugging process, repeated testing and correction of implementation issues helped us gain practical problem-solving experience.

ABSTRACT

Viragram is a university-focused social networking and campus community Android application developed with the idea of bringing student interaction and campus information into one digital platform. The project was motivated by the observation that students commonly use different applications for social posts, messaging, events, clubs and official announcements. Viragram attempts to combine these activities in a single university-oriented environment.

The application supports college-email-based registration, email verification, login, password recovery, a campus-oriented home feed, posts, image uploads, likes, comments, stories, follow and unfollow, search, profiles, notifications, messaging, dark mode and a College Hub. The College Hub connects students with events, clubs and announcements. An administrative system supports the creation and management of campus content such as events, clubs and announcements.

The Android application is implemented in Java with XML-based user interfaces. Firebase Authentication is used for identity and email/password authentication, while Cloud Firestore stores application data. ImageKit is used for media handling and Glide is used for image loading. A Node.js and Express backend provides backend functionality including ImageKit authentication. GitHub is used for source-code management and Render is used for backend deployment.

The project also demonstrates object-oriented programming through model classes, activity inheritance, method overriding and manager classes that abstract complex service operations. During development, practical problems were encountered and corrected, including Firestore permission errors, an unregistered admin activity, missing image URL propagation and inconsistent test-user data. Major application workflows were tested on an Android environment and the final project build was verified.

The completed project demonstrates how Android development, cloud services, media delivery, backend APIs, object-oriented design and software testing can be combined to create a campus-specific digital community platform.

TABLE OF CONTENTS

Chapter	Page
Abstract	1
1. Introduction	5
2. Brainstorming and Project Selection	12
3. Requirements and Feasibility	16
4. Development Methodology	23
5. Technologies and System Architecture	29
6. UML and System Design	37
7. Functional Modules	48
8. Working Product and Screenshots	57
9. Object-Oriented Programming	65
10. Testing and Validation	70
11. Problems Encountered and Solutions	76
12. Results, Limitations and Future Scope	81
References	86
Appendices	88

LIST OF FIGURES

Figure 1 — Viragram Use Case Diagram
Figure 2 — Viragram Class Diagram
Figure 3 — Object Diagram
Figure 4 — Create Post Sequence Diagram
Figure 5 — Registration and Login Activity Diagram
Figure 6 — Component Diagram
Figure 7 — Deployment Diagram
Figure 8 — Firestore Data Model
Figure 9 — System Architecture
Figure 10 — Admin Workflow
Figure 11 — Student User Journey
Figure 12 — Login Screen
Figure 13 — Home Feed
Figure 14 — College Hub
Figure 15 — Create New Post
Figure 16 — Profile
Figure 17 — Messages
Figure 18 — Notifications

LIST OF TABLES

Table 1 — Competitor comparison
Table 2 — Functional requirements
Table 3 — Non-functional requirements
Table 4 — Hardware and software requirements
Table 5 — OOP mapping
Table 6 — Core test cases
Table 7 — Problems and solutions
Table 8 — Future scope

CHAPTER 1 — INTRODUCTION

1.1 Background

University life produces a large amount of information and interaction every day. Students share photographs, communicate with classmates, discover events, join clubs, follow announcements and participate in activities. In practice, these activities are often distributed across different communication channels. Social media is useful for posts and interaction, messaging applications are useful for conversations, and institutional systems are useful for academic or official information. The separation makes campus information harder to discover in one place.

Viragram was conceived as a campus-focused social application rather than a general-purpose social network. Its central idea is to connect students with the people, content and activities that are relevant to their university. The project uses the familiar interaction model of a social feed while adding university-specific functions such as College Hub, clubs, events and announcements.

1.2 Motivation

The motivation for Viragram came from the need for a more connected digital campus experience. A student should not have to depend on multiple groups and applications simply to know what is happening on campus. A single platform can reduce fragmentation and make campus information more visible while still allowing students to interact socially.

1.3 Problem Statement

Students depend on multiple platforms for social networking, communication, events, clubs, announcements and media sharing. This creates information fragmentation. Important announcements may be missed, event information may be scattered, and students may not easily discover clubs or activities outside their immediate groups.

The problem addressed by Viragram can therefore be stated as follows: how can a university-focused Android application combine student social interaction with campus discovery and official campus information while maintaining a simple and familiar user experience?

1.4 Proposed Solution

Viragram provides a unified mobile platform where verified university users can create accounts, interact through posts and comments, discover other students, receive notifications, communicate through messages and access a College Hub containing events, clubs and announcements. Administrative users can publish and manage campus information.

1.5 Objectives

- Develop a university-focused Android social networking application.
- Restrict registration to the university email domain @unigoa.ac.in.
- Provide a social feed with posts, images, likes and comments.
- Provide student discovery through search, profiles and follow/unfollow.
- Provide College Hub features for events, clubs and announcements.
- Provide administrative controls for campus content.
- Integrate Firebase Authentication and Cloud Firestore.
- Integrate ImageKit for media handling and Glide for image loading.
- Demonstrate object-oriented programming and modular Android development.
- Test, debug and verify the major application workflows.

1.6 Scope

The scope of Viragram covers the Android client, authentication, social interaction, campus discovery, administrative content management and cloud data integration described in the project materials. The system is intended for a university community and uses the @unigoa.ac.in domain as the college-email restriction.

The current project scope includes registration, email verification, login, password recovery, home feed, posts, stories, likes, comments, follow/unfollow, search, profiles, notifications, messages, College Hub, events, clubs, announcements, admin functions and dark mode. Image uploads are handled through ImageKit and image URLs are stored with application records where applicable.

1.7 Significance

The significance of the project is both practical and academic. Practically, it provides a model for a university-specific community platform. Academically, it demonstrates Android programming, object-oriented design, database integration, authentication, backend communication, media handling, UI design, testing and debugging within one project.

1.8 Target Users

- Students who want to share and discover campus content.
- Students who want to discover events, clubs and announcements.
- Club administrators who publish club or event information.
- College administrators who manage official campus content.

1.9 Project Overview

Viragram is implemented as an Android application with Java and XML. The Android layer contains activities, fragments, adapters, model classes and utility/manager classes. Firebase Authentication manages user identity. Cloud Firestore provides the application data layer. Node.js and Express provide a backend service, particularly for ImageKit authentication, while Render hosts the production backend.

Project Item	Details
Project Name	Viragram
Tagline	Your Campus. Your Community.
Platform	Android
Language	Java
UI	XML
Package	com.example.viragram
Minimum SDK	API 24
Cloud	Firebase / Cloud Firestore
Authentication	Firebase Authentication, Email and Password
Media	ImageKit
Image Loading	Glide
Backend	Node.js + Express
Deployment	GitHub + Render

1.10 Project Deliverable

The final deliverable is a functional Android application supported by cloud services and a backend API. The project documentation includes requirements, design diagrams, working screenshots, testing evidence, problems encountered and the final results.

CHAPTER 2 — BRAINSTORMING AND PROJECT SELECTION

2.1 Initial Brainstorming

Brainstorming was used to identify a project idea that would be useful to university students and sufficiently demonstrate software engineering and object-oriented programming concepts. Several possibilities were considered before the campus social networking concept was accepted.

Idea	Main Purpose	Decision	Reason
Generic social media	General posts and profiles	Rejected	Too similar to existing platforms and lacked campus-specific purpose.
Attendance application	Student attendance tracking	Rejected	Mainly administrative and less focused on community interaction.
Complaint management	Student complaints and requests	Rejected	Useful but limited social and discovery functionality.
Event management	Create and discover events	Partially accepted	Useful, but insufficient as a complete social platform.
Discussion/forum	Student discussions	Partially accepted	Good interaction, but limited campus discovery.
Campus social network	Student interaction	Accepted	Strong balance of social and campus features.
Campus social + events + clubs	Unified campus platform	Final	Provides the broadest useful campus experience.

2.2 Evolution of the Idea

The idea evolved from a simple social application into a university-focused platform. The team first considered the basic social feed model: users, posts, likes and comments. During planning, it became clear that the project could provide greater value if campus information was integrated into the same experience.

The accepted concept therefore combined social networking with campus discovery. The resulting feature set included a feed for social content, profiles and search for student discovery, messaging and notifications for communication, and College Hub for events, clubs and announcements. Administrative roles were added so that official campus information could be published and managed.

2.3 Accepted and Rejected Ideas

- Accepted: campus social networking because it supports both student interaction and campus-specific information.
- Accepted: College Hub because it creates a central place for events, clubs and announcements.
- Accepted: image sharing because campus activities are naturally visual.
- Rejected: a completely generic social network because it would not have a strong university identity.
- Rejected as a standalone product: attendance management because it does not directly support community interaction.
- Rejected as the only module: event management because it would not demonstrate enough social functionality.

2.4 Competitor Analysis

The presentation material compares Viragram with Instagram, WhatsApp and LMS/college portals. The comparison focuses on the purpose of each platform rather than attempting to replace their strengths. Viragram is positioned as a campus-specific combination of social networking and official campus discovery.

Feature	Viragram	Instagram	WhatsApp	LMS / College Portal
Campus Focus	Yes	No	Partial	Yes
Social Feed	Yes	Yes	Limited	No
Campus Stories	Yes	Yes	Yes	No
Clubs & Events	Yes	Generic	Scattered	Partial
College Announcements	Yes	No	Scattered	Yes
Direct Messaging	Yes	Yes	Yes	Limited
College Hub	Yes	No	No	Partial
Admin Management	Yes	No	Limited	Yes

CHAPTER 3 — REQUIREMENTS AND FEASIBILITY

3.1 Functional Requirements

ID	Requirement	Description
FR01	Registration	Allow a student to create an account with full name, student ID, college email, course, academic year and password.
FR02	Email verification	Require the registered user to verify the college email before protected use.
FR03	Login	Authenticate users using email and password.
FR04	Password recovery	Allow users to request password recovery.
FR05	Home feed	Display campus-oriented posts and stories.
FR06	Create post	Allow a user to select an image, add a caption and publish a post.
FR07	Social interaction	Allow likes, comments, follow and unfollow.
FR08	Search	Allow discovery of users and campus content.
FR09	Profile	Display user information and posts.
FR10	Notifications	Show social and campus activity notifications.
FR11	College Hub	Display events, clubs and announcements.
FR12	Admin management	Allow authorized roles to create/manage campus content.
FR13	Media	Upload and load images using ImageKit and Glide.

3.2 Non-Functional Requirements

Category	Requirement
Usability	The application should use clear navigation, familiar social interaction patterns and readable screens.
Performance	Common screens should load without unnecessary processing and image loading should use a dedicated media service.
Reliability	Authentication and data operations should handle failures without crashing the application.
Security	Authentication and database access should be controlled using Firebase services and configured rules.
Maintainability	Code should be organized into activities, fragments, adapters, models and utility/manager classes.
Scalability	Cloud services and a separate backend provide a foundation for increasing users and content.
Compatibility	The application targets Android devices supported by the minimum API level specified by the project.
Availability	The backend is deployed through Render and the cloud services are externally hosted.
Privacy	User information and campus content should be accessed only through intended application flows.

3.3 User Requirements

- A valid university email address.
- Android device or emulator with internet access.
- Ability to register and verify email.
- Ability to upload/select images when using media features.
- Basic familiarity with social feed interaction.

3.4 Administrator Requirements

The administrator side requires role-aware access so that campus content management functions are not treated as ordinary student functions. The project materials identify roles including student, club_admin and college_admin.

- Create and manage events.
- Create and manage clubs.
- Create and manage announcements.
- Publish campus information for College Hub.
- View dashboard information and metrics where provided by the application.
- Use role checking before exposing administrative functions.

3.5 Hardware and Software Requirements

Type	Requirement
Development OS	Windows / Linux environment capable of running Android Studio and Node.js.
IDE	Android Studio
Language	Java
UI	XML
Android Minimum SDK	API 24
Cloud	Firebase Authentication and Cloud Firestore
Media	ImageKit
Image Loading	Glide
Backend	Node.js + Express
Source Control	Git / GitHub
Deployment	Render
Testing	Android emulator and/or Android device

3.6 Feasibility Study

Technical Feasibility

The project is technically feasible because the selected technologies provide the required building blocks. Android Studio supports Java-based Android development, Firebase provides authentication and cloud data, ImageKit provides media handling, and Node.js/Express can expose backend endpoints. The separation between the Android client, Firebase services and backend makes the system easier to understand and extend.

Economic Feasibility

The project uses widely available development tools and cloud services. The development environment can be established without specialized hardware beyond a normal development computer and an Android test environment. Actual production cost depends on service usage and deployment plans, but the architecture is suitable for an academic prototype.

Operational Feasibility

The application follows familiar social-media interaction patterns, which lowers the learning curve for students. Administrative functions are separated from the normal student experience, while College Hub provides a direct route to campus information.

Schedule Feasibility

The development was organized iteratively. Authentication and basic navigation were implemented first, followed by feed and social features, media integration, College Hub, administration, testing and deployment.

CHAPTER 4 — DEVELOPMENT METHODOLOGY

The development followed an iterative, Agile-oriented approach. Instead of attempting to complete the entire system at once, the application was developed in feature groups. Each group was implemented, tested and refined before moving to the next major area.

Phase	Main Activity	Outcome
1	Idea and brainstorming	Campus social platform selected.
2	Requirement analysis	Functional and non-functional requirements defined.
3	UI planning	Screens and navigation structure planned.
4	Firebase setup	Authentication and Firestore configured.
5	Authentication	Registration, verification, login and recovery implemented.
6	Home feed	Posts, stories and feed structure developed.
7	Media	ImageKit and Glide integration added.
8	Social interaction	Likes, comments, follow and search implemented.
9	College Hub	Events, clubs and announcements integrated.
10	Admin portal	Role-aware campus content management added.
11	Testing	Major workflows tested.
12	Bug fixing	Permission, manifest and image-passing issues corrected.
13	Deployment	Backend deployed using Render and source managed through GitHub.
14	Final verification	Build and major workflows verified.

4.1 UI and Navigation Planning

The interface follows a mobile-first structure with a persistent bottom navigation area for major student functions. The supplied screens show a consistent purple visual identity, a simple top bar and familiar social controls. The home screen includes stories, a feed and interaction controls. Separate screens are provided for College Hub, post creation, profile, messages and notifications.

The UI planning also considered error and empty states, readable touch targets and dark mode. The Create New Post screen, for example, provides image selection, a demo-image option, caption entry, location, tagging and story-sharing controls.

4.2 Firebase Setup

The Firebase project used by Viragram is identified as viragram-78116. Firebase Authentication is configured for email/password sign-in. The university email restriction is implemented using the @unigoa.ac.in domain. Cloud Firestore stores application data such as users, posts, stories, comments, events, announcements and clubs.

4.3 Authentication Development

The authentication flow includes registration, email verification, login, password recovery and logout/session handling. The project structure identifies LoginActivity, RegisterActivity, VerifyEmailActivity and ForgotPasswordActivity, while FirebaseAuthManager provides a wrapper around authentication operations.

4.4 Feed and Social Feature Development

The home feed was developed as the main social surface. The project includes model and adapter classes for posts, stories and comments. Users can create posts with captions and images, view posts, like posts and add comments. Follow and unfollow functions extend the social graph, while search and profiles provide discovery.

4.5 Media Integration

ImageKit was selected for media handling. The Android application requests an ImageKit authentication response from the Node.js/Express backend through /api/imagekit/auth. After upload, the resulting image URL is associated with the relevant application record. Glide is used to load remote images in Android screens.

4.6 College Hub and Admin Development

College Hub was developed as the campus-information layer. Administrative users can create events, announcements and clubs. The project uses “Viragram Tech Fest 2026” as an example event. The intended flow is administrator creation, Firestore storage, College Hub retrieval and student discovery.

4.7 Testing and Debugging Cycle

Testing was performed throughout development rather than only at the end. When a feature failed, the team traced the failure to the relevant layer. For example, a Firestore permission error was investigated through authentication and security rules; an admin event crash was traced to Android activity registration; and blank images were traced through adapter-to-detail-screen URL passing.

4.8 Deployment Workflow

The project uses GitHub for source-code/version management and Render for backend deployment. The backend service is identified as viragram-backend, with a health endpoint at /health. The production backend is viragram-backend.onrender.com. The development flow can therefore be summarized as local development → testing → GitHub → Render deployment → production backend.

The project materials do not establish a full automated CI/CD pipeline, so this report does not claim one.

CHAPTER 5 — TECHNOLOGIES AND SYSTEM ARCHITECTURE

5.1 Java and Android

Java is the primary programming language of the Viragram Android application. It supports the object-oriented structure of the project through classes, encapsulation, inheritance and method overriding. Android Studio is used as the development environment, while XML is used for user-interface layouts.

The package is com.example.viragram and the minimum Android SDK is API 24. The project is organized into activities, fragments, adapters, models and utility/manager classes. This organization separates screen behavior, data representation and reusable application logic.

5.2 Firebase Authentication

Firebase Authentication provides the identity layer. Users register with email and password, receive email verification, log in after verification, and can use password recovery. The college-email rule uses the @unigoa.ac.in domain to keep the application focused on the university community.

5.3 Cloud Firestore

Cloud Firestore is the primary application data store. The project identifies collections such as users/{uid}, posts/{postId}, stories/{storyId}, comments under posts, events/{eventId}, announcements/{announcementId} and clubs/{clubId}. Firestore listeners can provide updated data to the Android interface without requiring a full manual refresh.

5.4 ImageKit and Glide

ImageKit is used for media/image handling. The separation of media storage from application data keeps large image files out of ordinary Firestore documents. The backend exposes /api/imagekit/auth so that the Android application can obtain the authentication information needed for the upload flow.

After an image is uploaded, its URL can be stored with the corresponding user, post or event data. Glide is then used by Android components to retrieve and display remote images efficiently, with fallback handling where required.

5.5 Node.js and Express

Node.js and Express provide the Viragram backend service. The backend is not presented as a replacement for Firebase Authentication or Firestore. Instead, it provides backend functionality required by the application, especially ImageKit authentication. This distinction is important because the Android application directly uses Firebase Authentication and Firestore while also communicating with the Node/Express service.

5.6 GitHub and Render

GitHub is used for source-code/version management. The project materials identify a private repository named Viragram-Backend. Render is used to deploy the backend service. The production service is identified as viragram-backend.onrender.com and includes a /health endpoint intended to verify backend availability.

5.7 System Architecture

The system architecture contains four principal technical areas: the Android application, Firebase services, the Node.js/Express backend and ImageKit. The Android application communicates directly with Firebase Authentication and Cloud Firestore. It also communicates with the Node/Express backend for backend functions such as ImageKit authentication. The backend then interacts with ImageKit for media operations.

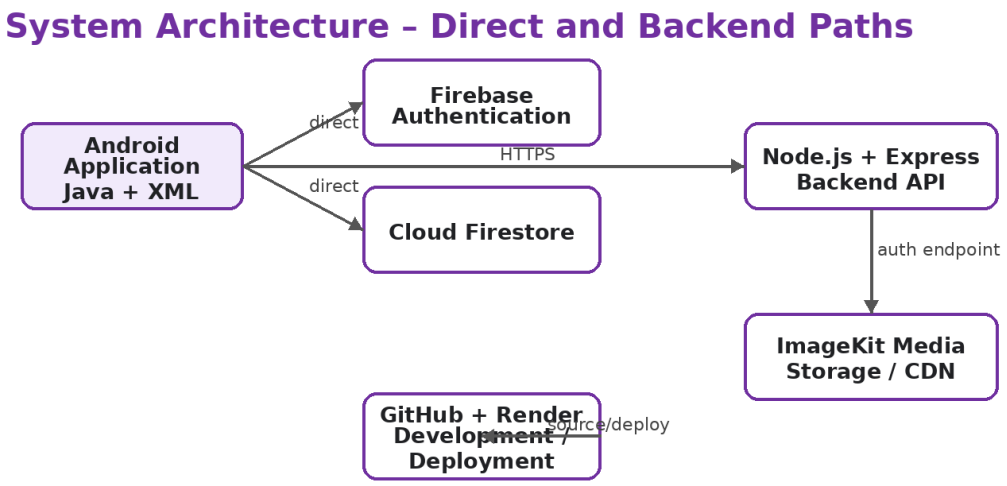


Figure 9: System Architecture — Android, Firebase, Backend and ImageKit

CHAPTER 6 — UML AND SYSTEM DESIGN

UML diagrams are used to represent the structure and behavior of Viragram in a form that is easier to understand than source code alone. The following diagrams are based on the implemented features and the project structure supplied for the application.

6.1 Use Case Diagram

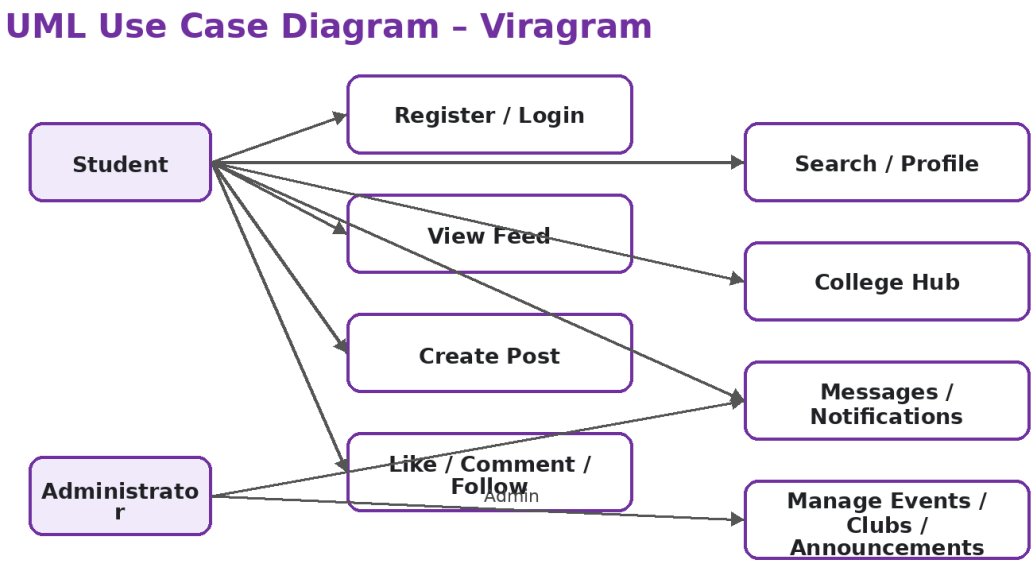


Figure 1: UML Use Case Diagram — Viragram

The student actor performs registration, login, feed interaction, post creation, social interaction, search, profile management, College Hub discovery, messaging and notifications. The administrator actor manages campus events, announcements and clubs.

6.2 Class Diagram

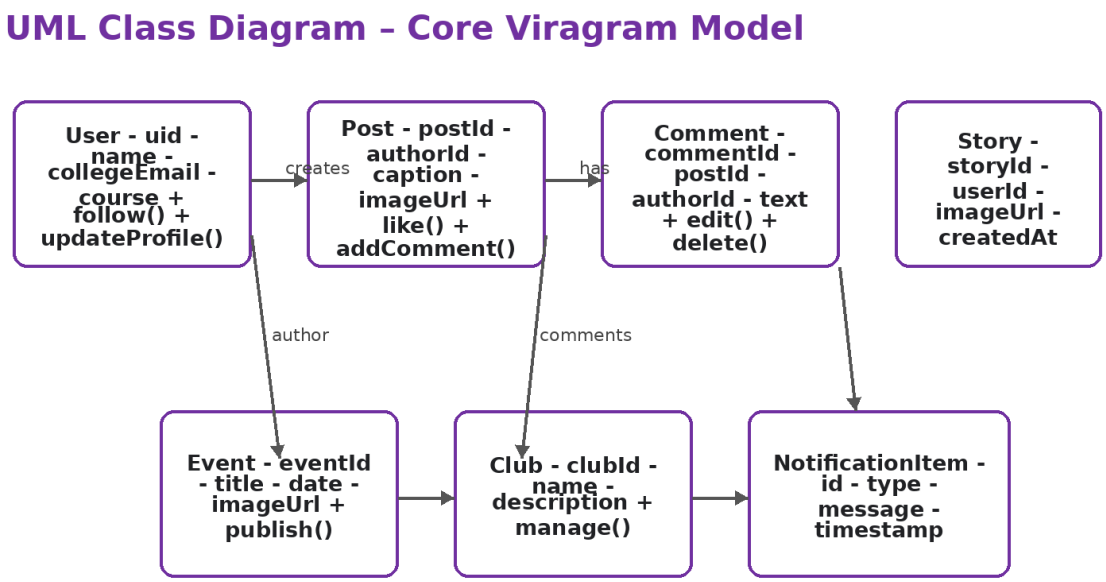


Figure 2: UML Class Diagram — Core Viragram Model

The class model represents the major domain objects identified in the project: User, Post, Comment, Story, Event, Club and NotificationItem. Relationships show typical ownership or reference patterns such as a user creating posts and comments belonging to posts. The diagram is a conceptual model of the application classes and Firestore entities described in the project materials.

6.3 Object Diagram

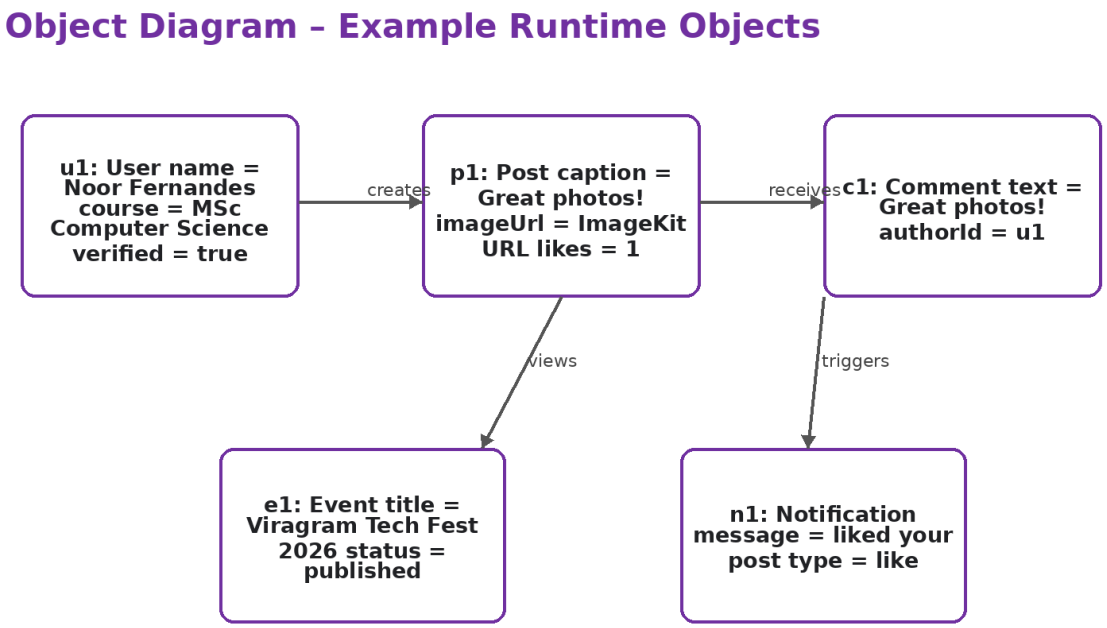


Figure 3: Object Diagram — Example Runtime Objects

The object diagram shows example runtime instances rather than class definitions. For example, a User object may create a Post object, a Comment object can reference that post, and a Notification object can represent a social interaction. The values are illustrative examples based on the supplied working screens and project terminology, not a claim about a permanent database record.

6.4 Sequence Diagram

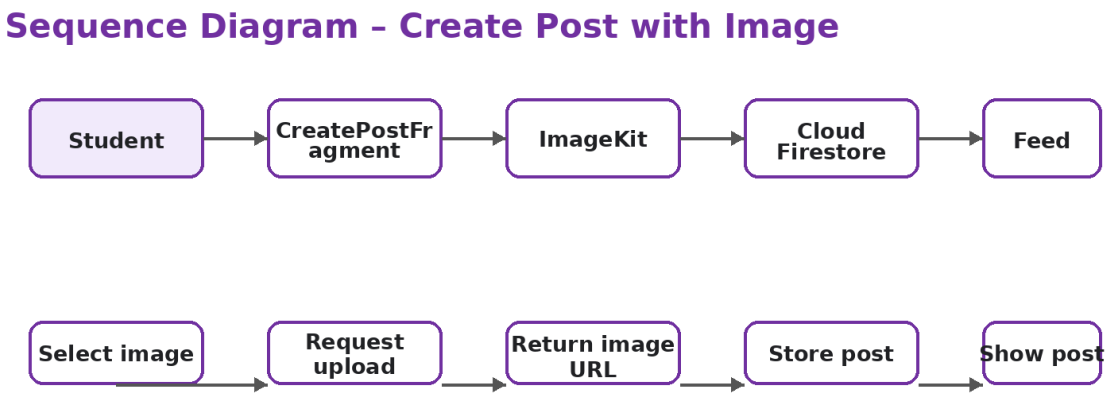


Figure 4: Sequence Diagram — Create Post with Image

The sequence begins when the student selects an image in CreatePostFragment. The application requests upload authentication from the backend, sends the media to ImageKit, receives an image URL, stores the post metadata in Firestore and finally displays the post in the feed. This sequence reflects the separation between media delivery and application data.

6.5 Activity Diagram

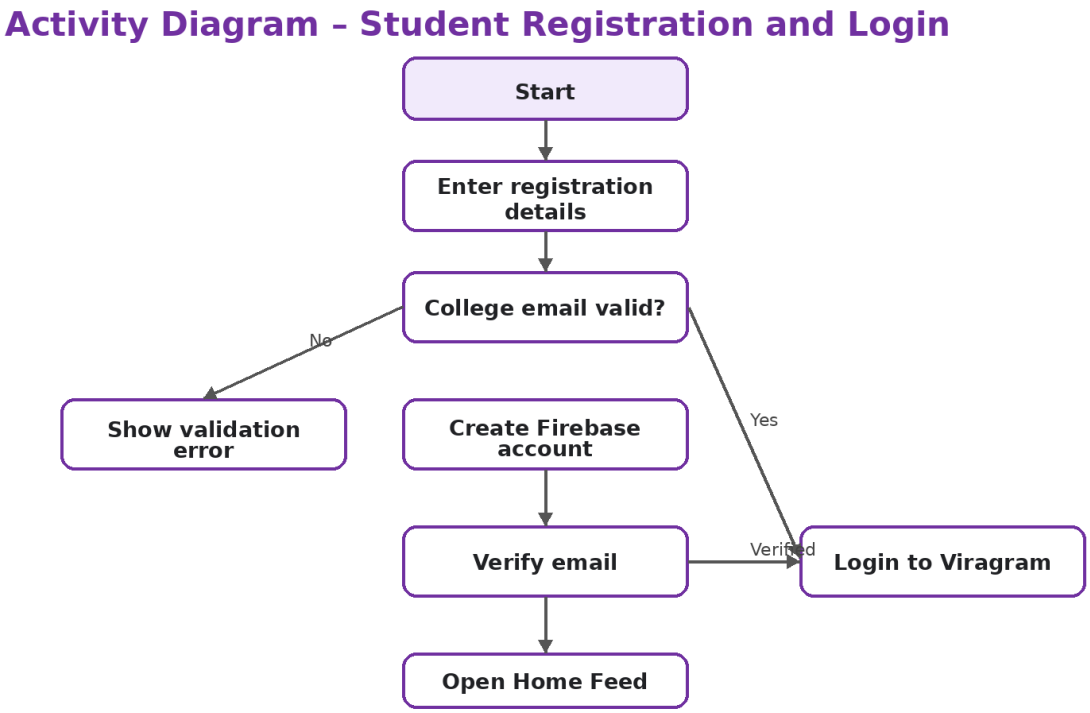


Figure 5: Activity Diagram — Registration and Login

The activity diagram shows the main authentication path. The user enters registration information, the college-email condition is checked, the Firebase account is created, the email is verified and the user can then log in and open the home feed. Invalid input is returned to the user for correction.

6.6 Component Diagram

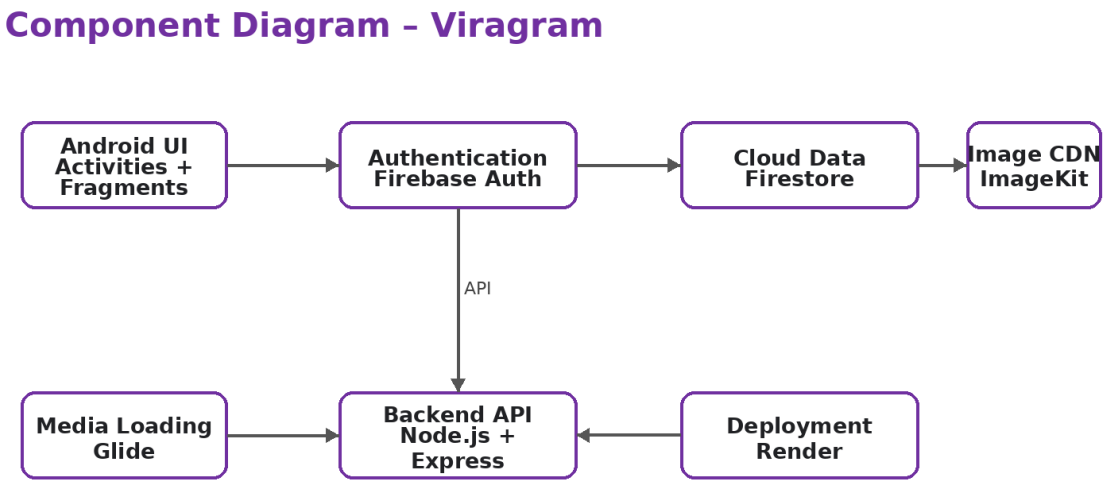


Figure 6: Component Diagram — Viragram

The component diagram groups the system into Android UI, Firebase Authentication, Firestore, backend API, ImageKit, Glide and deployment/source components. The purpose is to show responsibility boundaries rather than individual Java files.

6.7 Deployment Diagram

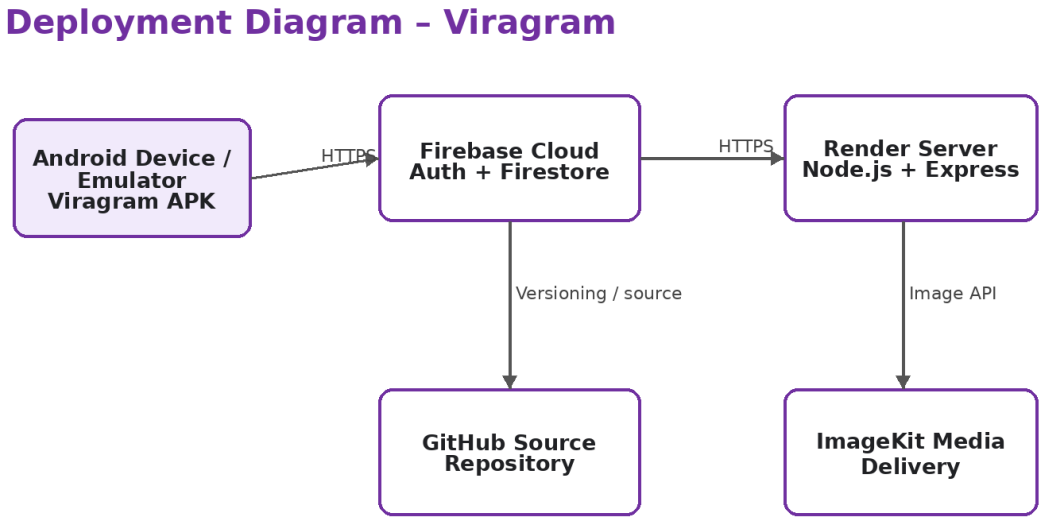


Figure 7: Deployment Diagram — Viragram

The deployment diagram shows an Android device or emulator communicating with Firebase and the deployed backend. The Render-hosted backend communicates with ImageKit, while GitHub represents the source repository used during development and deployment.

6.8 Firestore Data Model

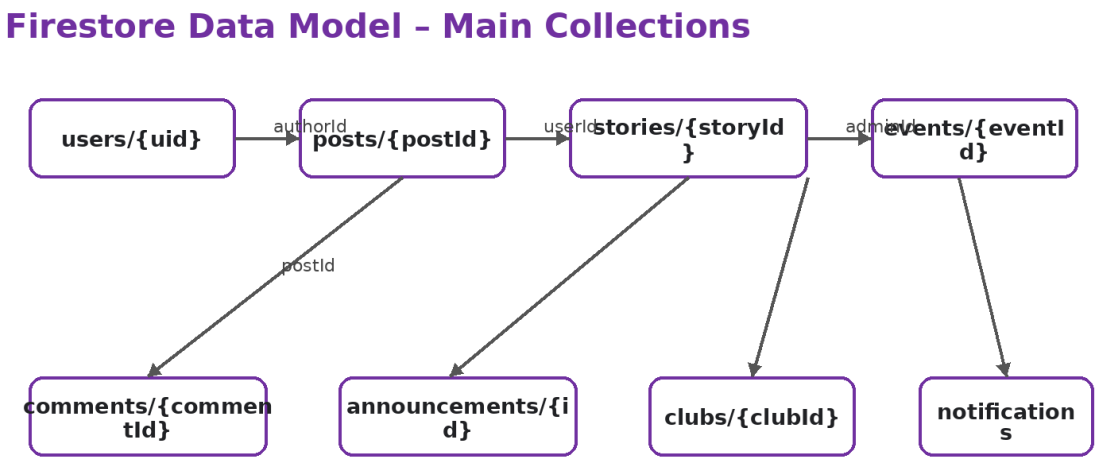


Figure 8: Firestore Data Model — Main Collections

The data model represents the main Firestore collections identified by the project. Users provide identity information, posts and stories hold social content, comments belong to posts, and events, clubs and announcements provide campus information. Notification records support activity feedback.

6.9 Admin Workflow

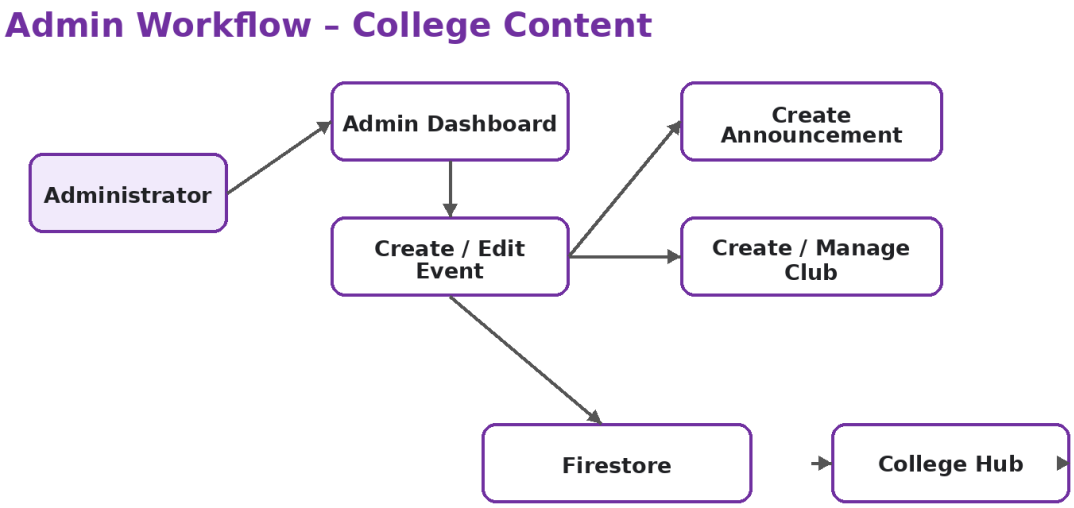


Figure 10: Admin Workflow — College Content

The administrator starts from the dashboard and creates or manages events, announcements and clubs. These records are stored in Firestore and then surfaced through the College Hub for students. The example event “Viragram Tech Fest 2026” demonstrates this campus-content flow.

6.10 Student User Journey

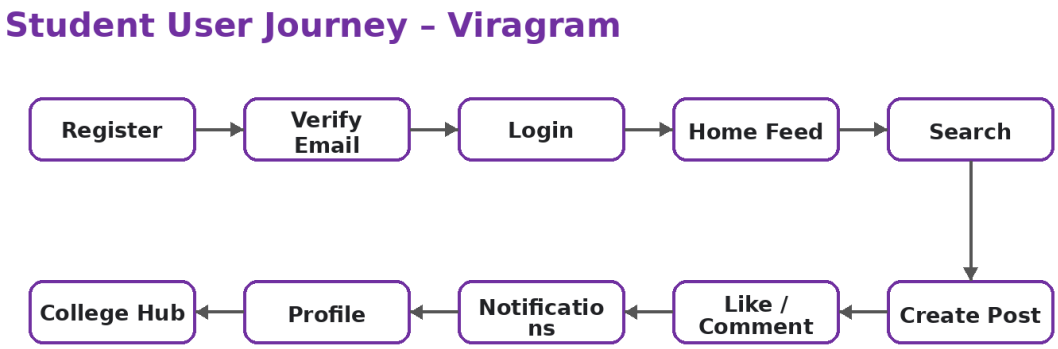


Figure 11: Student User Journey — Viragram

The student journey begins with university registration and email verification, followed by login and use of the social feed. Students can search, create posts, interact with other users, receive notifications, manage their profile and discover campus information through College Hub.

CHAPTER 7 — FUNCTIONAL MODULES

7.1 Registration and Email Verification

Registration collects full name, student ID, college email, course, academic year and password. The college email requirement is an important part of the application's campus identity. Firebase Authentication manages the account while the application can store additional profile data in Firestore. Email verification provides a second step before protected use.

7.2 Login and Password Recovery

The login screen provides fields for college email and password, a password-visibility control and a Forgot password option. Firebase Authentication handles credential verification. Password recovery provides a controlled way for a user to regain access without creating a new account.

7.3 Home Feed and Stories

The home feed is the primary social screen. The supplied screenshot shows a top navigation bar, a horizontal story area, a post header with user information, a large image, reaction controls, caption information and comment access. This design gives the application a familiar social interaction model while keeping the campus identity visible.

Stories provide a short-form visual surface above the feed. The project includes Story and StoryAdapter classes, indicating that stories are treated as a separate model and UI component.

7.4 Create Post and Image Upload

The Create New Post screen allows a student to select a campus photo or use a demo photo, enter a caption, specify a location, tag people and optionally share to a campus story. The post is intended to store metadata and the uploaded image URL before appearing in the feed.

The media flow uses ImageKit for upload and Glide for displaying remote images. This separation was important during debugging because missing URL propagation could cause a detail screen to show a blank image.

7.5 Likes, Comments, Follow and Search

Social interaction includes likes and comments on posts. Users can follow and unfollow other students, which supports a student-centric social graph. Search provides a discovery route for students and campus content. These features move the application beyond a simple announcement portal and make it a social community platform.

7.6 Profile Management

The profile screen shows the user's profile photo, name, username, course information, post/follower/following counts and a grid of posts. The supplied profile screenshot also shows controls for editing and sharing the profile. This gives each student a personal space for identity and shared content.

7.7 Messages and Notifications

The messages screen provides a list of conversations with users and clubs. It includes search and recent conversation previews. The notifications screen groups activity by time period and shows actions such as likes, comments, follows and campus updates.

The project includes Conversation, ChatMessage and NotificationItem model classes, along with LocalMessageManager and ViragramNotificationManager utility/manager classes. These components provide a structured way to represent and display communication and activity data.

7.8 College Hub

College Hub is one of the major differentiating features of Viragram. The supplied screen contains a campus banner, filter chips such as All, Trending, Events and Clubs, a trending activities area and an upcoming events section. The screen demonstrates how social activity and official campus discovery can coexist.

The example “Viragram Tech Fest 2026” shows an event card with an image, date, category, description and organizer. The expected data flow is Admin → Firestore → College Hub → Student, with event details available through the event detail screen.

7.9 Admin Portal

The admin system separates campus-content management from student social interaction. The identified roles include student, club_admin and college_admin. Administrative activities include an admin dashboard, event management, event creation, announcement creation and club management.

Admin Function	Purpose
Dashboard	Provide an entry point to management functions and dashboard information.
Events	Create, edit and manage campus events.
Create Event	Enter event details and publish them to the campus data layer.
Announcements	Create official campus updates.
Clubs	Create and manage campus clubs.
Role Checking	Expose administrative functionality only to appropriate roles.
College Hub Synchronization	Make managed campus information discoverable by students.

7.10 Dark Mode and UI Consistency

Dark mode is included as a UI feature to improve viewing flexibility. The application also uses a consistent purple accent, rounded cards, clear icons and bottom navigation. The project development included UI/UX polishing such as toolbar consistency, bookmark controls, animations and empty-state handling.

The screenshots supplied with the project show that the final interface is intentionally simple. The design uses a small number of strong visual accents rather than dense screens, which helps students move between social and campus functions.

CHAPTER 8 — WORKING PRODUCT AND SCREENSHOTS

The following screenshots are actual application screens supplied with the project. They are included as evidence of the implemented user interface. Each screenshot is discussed in terms of the function it demonstrates.

Figure 12: Login Screen

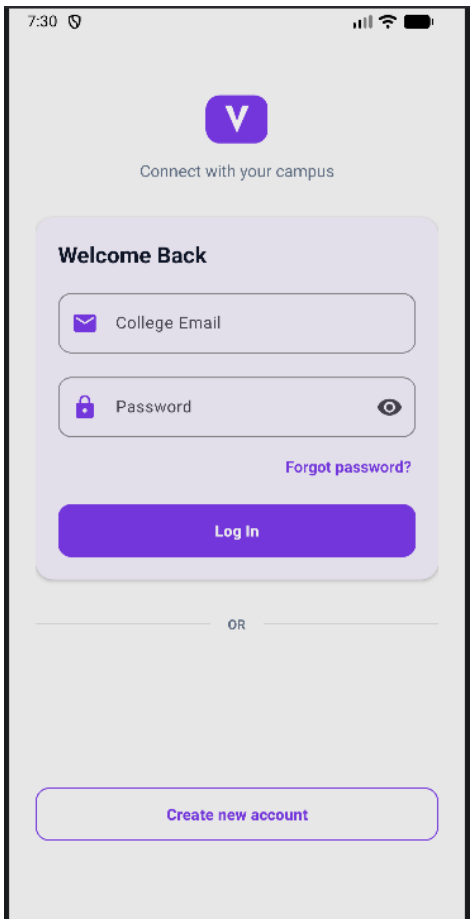


Figure 12: Login Screen — Firebase Authentication Entry Point

The Login screen provides college email and password fields, password visibility, password recovery and account creation. It represents the primary authentication entry point for returning students.

The login screen is designed for students who already have a verified university account. The form keeps the two main inputs visible: college email and password. The password field also includes a visibility control so that the user can check the entered value before submitting.

The screen provides a Forgot password option for account recovery and a Create new account option for new students. The purple action button is used as the main call to action and follows the same visual style used throughout the application.

From the implementation side, LoginActivity works with the Firebase authentication layer. The application checks the login details and then continues to the main application after successful authentication.

Figure 13: Home Feed



Figure 13: Home Feed — Stories and Social Post

The Home Feed shows stories, a campus-oriented post, like/comment/share controls and post information. It is the main social interaction surface of Viragram.

The Home Feed is the main screen used after login. The top area contains stories, followed by a post card containing the user identity, follow control, post image, reaction controls, caption and comment information.

The feed is intended to make campus interaction simple. A student can view campus-related posts, like a post, open comments, share content and move to other sections using the bottom navigation bar.

The implementation uses Java model and adapter classes for posts, stories and comments. This separates the data representation from the Android UI and makes the feed easier to maintain.

Figure 14: College Hub

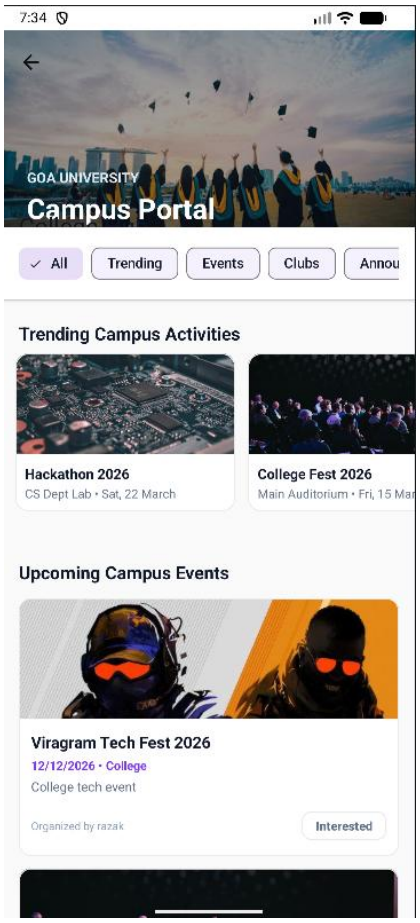


Figure 14: College Hub — Campus Events and Activities

The College Hub screen presents campus activity categories, trending activities and upcoming events. It demonstrates the discovery layer of the application.

College Hub is the campus information area of Viragram. The screen uses a large campus banner followed by filters such as All, Trending, Events and Clubs. This helps a student move from the general feed to more specific campus information.

The page displays trending campus activities and upcoming events. Each event card contains an image, title, date, category and a short description. The Interested button gives the user a simple way to interact with an event.

The College Hub is important because it gives Viragram a university identity instead of functioning only as a normal social media application. Events, clubs and announcements can be presented together in one place.

Figure 15: Create New Post

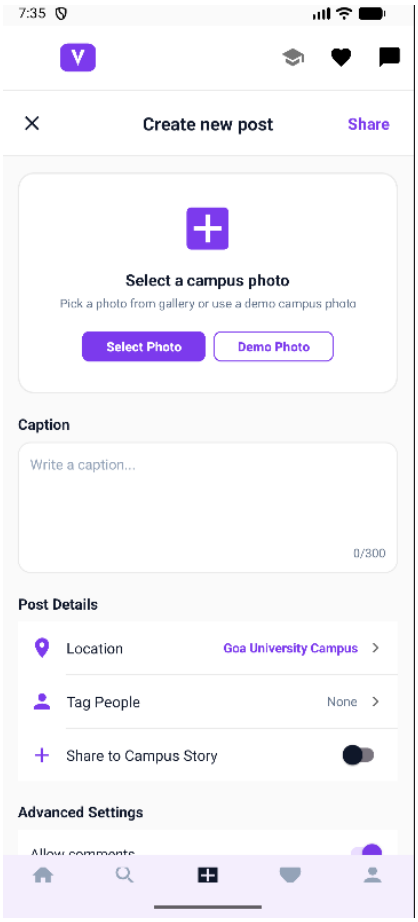


Figure 15: Create New Post — Image and Caption Entry

The Create New Post screen provides photo selection, caption entry, location, tagging and campus-story sharing options. It demonstrates the post creation workflow.

The Create New Post screen is used when a student wants to publish new campus content. The screen provides photo selection, a caption field, location information, tagging and an option to share the post to a campus story.

The screen also contains advanced settings for controlling post behaviour. The layout keeps the image selection area at the top so that the user can first choose the main content and then enter the supporting details.

In the project implementation, image information and post metadata are handled separately. The selected image can be uploaded through the media service and the resulting URL can be associated with the post data.

Figure 16: Profile

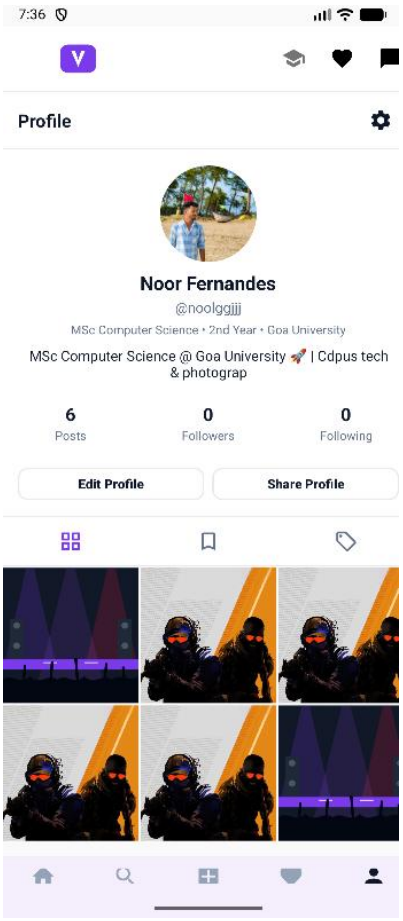


Figure 16: Profile — Student Identity and Post Grid

The Profile screen displays the student's identity, course information, counts and post grid. It also provides Edit Profile and Share Profile controls.

The Profile screen represents the identity of a student inside Viragram. It shows the profile photo, name, username, course and university information along with the number of posts, followers and following.

The screen provides Edit Profile and Share Profile actions. Below the profile information, a tab area and post grid allow the student to view the content associated with the account.

The profile module connects the social identity of the user with the content created by that user. This makes it possible to move from a feed post to a student's profile and then browse the student's published posts.

Figure 17: Messages

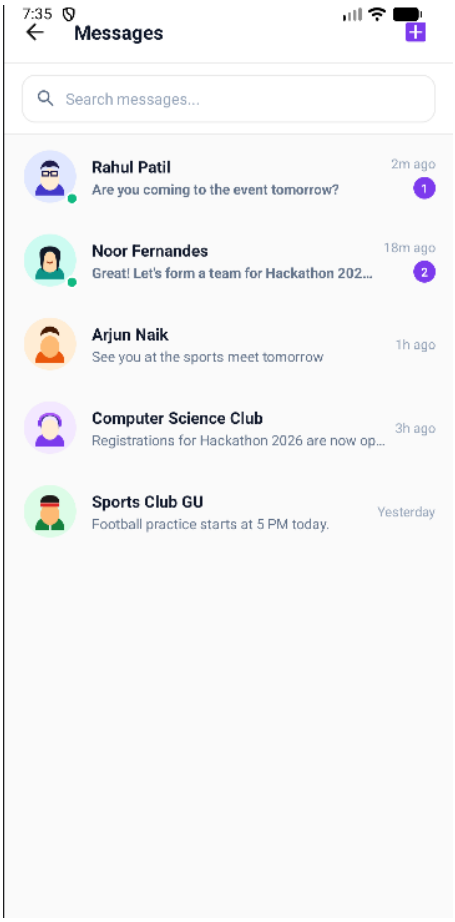


Figure 17: Messages — Conversation List

The Messages screen shows recent conversations with students and campus clubs, along with search and unread indicators.

The Messages screen provides a simple conversation list. A search field is available at the top and each conversation displays the contact name, recent message preview, time and unread indicator where applicable.

The screen supports conversations with both individual students and campus clubs. This is useful for event coordination, student communication and club-related discussions without leaving the Viragram application.

The project contains Conversation and ChatMessage model classes together with message-management utilities. The UI is therefore separated from the message data and conversation representation.

Figure 18: Notifications

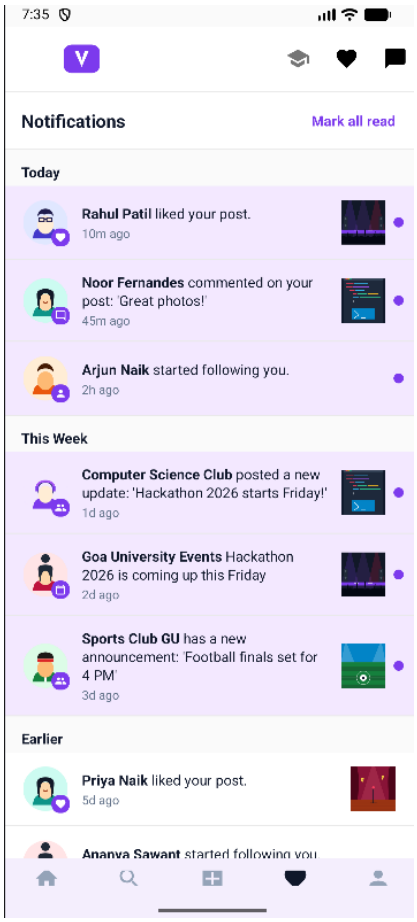


Figure 18: Notifications — Social and Campus Activity

The Notifications screen groups recent activity such as likes, comments, follows and campus announcements. It also provides a Mark all read control.

The Notifications screen brings social and campus activity together in one place. Examples shown in the supplied screen include likes, comments, new followers and campus announcements.

Notifications are grouped using time-based sections such as Today, This Week and Earlier. This makes recent activity easier to identify without showing every event as an unrelated item.

The screen also provides a Mark all read action. NotificationItem and the notification manager are used in the project to represent and handle notification information.

CHAPTER 9 — OBJECT-ORIENTED PROGRAMMING IN VIRAGRAM

Object-oriented programming is central to Viragram because the Android application is written in Java and organized around classes representing users, posts, stories, comments, events, clubs, notifications and messages. The project also uses managers and utilities to keep service-specific logic separate from screen code.

9.1 Encapsulation

Encapsulation means keeping related data and operations together inside a class and controlling how the data is accessed. Viragram model classes such as User, Post, Event and Comment represent application data. Where fields are private, getters and setters or other methods provide controlled access. This keeps screen code from depending directly on the internal representation of each object.

9.2 Inheritance

Android activities inherit from Android framework classes, and the project materials also describe a BaseActivity approach for sharing common behavior. Inheritance allows common functionality to be placed in a parent class while individual activities implement their own screen-specific behavior. This reduces duplication and supports a consistent application structure.

9.3 Polymorphism

Polymorphism allows an object to be used through a common inherited interface while the subclass supplies specialized behavior. In Viragram, Android lifecycle methods such as onCreate() are overridden by activities to initialize their particular screens. The important point is that the polymorphic behavior comes from overriding inherited methods; the method name alone is not the concept.

9.4 Abstraction

Abstraction hides implementation details behind simpler operations. FirebaseAuthManager and Firestore-related manager classes provide an example of this design idea. A screen can request an authentication or data operation through a manager instead of implementing every service-specific detail inside the UI layer. This makes the application easier to maintain.

9.5 OOP Mapping

Concept	Project Example	Explanation
Encapsulation	User, Post, Event, Comment	Data and related operations are organized inside model classes.
Inheritance	Activities / BaseActivity	Common Android behavior can be reused through parent classes.
Polymorphism	Overridden onCreate() and lifecycle methods	Subclasses provide specialized behavior for inherited methods.
Abstraction	FirebaseAuthManager / data managers	Complex service operations are exposed through simpler methods.

CHAPTER 10 — TESTING AND VALIDATION

10.1 Testing Strategy

Testing focused on the major user and administrator workflows. The supplied presentation states that Viragram was tested on an Android emulator with major workflows including registration and login, university email verification, create post, ImageKit upload, Firebase/Firestore integration, likes and comments, follow/unfollow, search and profiles, College Hub, admin events, announcements and clubs, and dark mode.

Testing Type	Purpose
Functional	Verify that each feature performs its intended operation.
Positive	Verify valid inputs and expected successful flows.
Negative	Verify invalid inputs and controlled error handling.
Integration	Verify Android-to-Firebase/backend/media interactions.
UI	Verify navigation, screen layout and visible controls.
Authentication	Verify registration, verification, login and recovery.
Firestore	Verify reads/writes and access rules.
Image Upload	Verify media upload and URL storage/display.
Admin	Verify role-specific management functions.
Regression	Re-test important workflows after fixes.

10.2 Core Test Cases — Authentication and Posts

ID	Module	Scenario	Expected Result	Status
TC01	Registration	Valid @unigoa.ac.in registration	Account created and verification initiated	Tested
TC02	Registration	Invalid/non-college email	Validation prevents inappropriate registration	Tested
TC03	Verification	Verify registered email	User becomes eligible for protected use	Tested
TC04	Login	Valid credentials	Home screen opens	Tested
TC05	Password	Forgot password	Password recovery request is initiated	Tested
TC06	Create Post	Valid caption and image	Post is created and appears in feed	Tested
TC07	Image Upload	Upload image through ImageKit	Image URL is returned and used by the application	Tested
TC08	Like	Tap like on a post	Like state updates	Tested
TC09	Comment	Add valid comment	Comment appears on the post	Tested
TC10	Search	Search for a user/content	Relevant result is displayed	Tested
TC11	Follow	Follow another user	Follow state updates	Tested
TC12	Unfollow	Unfollow followed user	Follow state is removed	Tested

10.3 Core Test Cases — Campus and Admin

ID	Module	Scenario	Expected Result	Status
TC13	Story	Create/view story	Story appears in the story area	Tested
TC14	Notifications	Generate social/campus activity	Notification is displayed	Tested
TC15	Profile	Open profile	Profile information and post grid display	Tested
TC16	Admin Role	Open admin functions with authorized role	Admin functions are available	Tested
TC17	Create Event	Create Viragram Tech Fest 2026	Event is stored and available to College Hub	Tested
TC18	Announcement	Create announcement	Announcement is published to campus content	Tested
TC19	Club	Create/manage club	Club information becomes available	Tested
TC20	College Hub	Open Hub	Events, clubs and announcements are discoverable	Tested
TC21	Event Detail	Open event detail	Event information and image display	Tested after fix
TC22	Post Detail	Open post detail	Post information and image display	Tested after fix
TC23	Dark Mode	Switch theme	Interface changes to	Tested

			dark theme	
TC24	Logout	Logout current user	Session is ended and login screen becomes available	Tested

The status wording above is based on the supplied project presentation and development history. Where a feature was specifically described as fixed and tested, that is noted. No numerical performance claims are added because the project materials do not provide benchmark measurements.

10.4 Integration and Regression Testing

Integration testing focused on the points where independent components meet. Authentication was checked together with protected application access. Firestore was checked together with post creation and campus-content retrieval. ImageKit was checked together with the Android upload interface and Glide-based display. The backend health endpoint was also used during investigation of connectivity-related behavior.

Regression testing was especially important after the bug fixes. After correcting Firestore access, the post creation workflow was retested. After registering the missing admin activity, the create-event flow was retested. After correcting image URL propagation, post and event detail screens were retested. This ensured that a local fix did not leave the larger workflow broken.

CHAPTER 11 — PROBLEMS ENCOUNTERED AND SOLUTIONS

11.1 Firestore PERMISSION_DENIED

During post creation, the application could reach the point of creating the post but Firestore rejected the write with a PERMISSION_DENIED error. This showed that the problem was not simply the UI button or the post form; access to the database operation also depended on authentication and Firestore rules.

The issue was investigated through the Firestore security configuration and authenticated access path. The rules were corrected and published, and the post-creation workflow was tested again. The supplied project presentation records the final result as successful post uploads after the Firestore security rules were corrected.

11.2 Admin Event Screen Crash

The admin event creation screen crashed when the "+ Event" action was selected. The cause was identified as a missing registration of AdminCreateEventActivity in AndroidManifest.xml. Because Android activities must be declared when required by the application configuration, the navigation target could not be launched correctly.

The solution was to register the activity in AndroidManifest.xml, rebuild the application and test the create-event flow again. This is a useful example of how a runtime UI problem can originate from application configuration rather than from the visible screen itself.

11.3 Blank Post and Event Images

The Post Detail image appeared blank because the image URL was not being passed correctly to PostDetailActivity. The correction updated ProfilePostAdapter so that post.getImageUriString() was passed through PostDetailActivity.EXTRA_IMAGE_URL. The detail activity then used Glide to load the remote image.

A similar problem occurred on Event Detail. SearchEventAdapter and TrendingCampusAdapter were updated to pass event.getImageUrl(), after which EventDetailActivity could load the remote image with appropriate fallback handling. These issues demonstrated the importance of tracing data across adapters, intents and detail screens.

11.4 User Identity and Login Connectivity Investigation

During development, different test-user identities such as Rahul Patil and Noor Fernandes appeared in the application. This was treated as a test-data and identity-consistency issue rather than as a functional feature. Consistent test accounts are important when verifying profile, post ownership and social interactions.

Another issue displayed the message “Network connection error. Please check your internet connection.” The project materials state that the Render health endpoint was tested successfully, so the investigation did not simply assume that the backend was unavailable. Checks included Firebase Authentication, Firebase configuration, INTERNET permission, authentication initialization, Firebase exception/error mapping, emulator connectivity and Android configuration.

The supplied materials do not provide enough evidence to state a single final root cause for this login message. Therefore, this report records the investigation without inventing a final cause.

CHAPTER 12 — RESULTS, LIMITATIONS AND FUTURE SCOPE

12.1 Final Result

The final Viragram system brings university social networking and campus information together in one Android application. The project materials record that the major workflows were tested and that the final build was verified successfully. The completed feature set demonstrates registration and authentication, social feed interaction, media handling, profiles, messaging, notifications, College Hub, administrative content management and dark mode.

Area	Final Outcome
Authentication	Registration, verification, login and password recovery implemented.
Social	Posts, stories, likes, comments, follows and profiles implemented.
Discovery	Search and College Hub support student and campus discovery.
Campus Content	Events, clubs and announcements are managed through the admin side.
Media	ImageKit upload and Glide-based remote image loading implemented.
Cloud	Firebase Authentication and Cloud Firestore integrated.
Backend	Node.js / Express backend deployed using Render.
OOP	Java-based model, activity and manager structure demonstrates OOP.
Testing	Major application workflows tested and bugs corrected.

12.2 Limitations

- Internet connectivity is required for cloud-backed functionality.
- Firebase and ImageKit availability directly affects corresponding application features.
- Render deployment characteristics may affect backend response time depending on the hosting plan.
- Production-scale load testing was not established by the supplied project evidence.
- Content moderation is limited compared with a large production social platform.
- Advanced real-time communication features remain suitable areas for future development.
- The project has not been presented as a full public Play Store deployment.

12.3 Future Scope

- Real-time Firebase chat improvements and richer messaging.
- Push notifications for social and campus activity.
- Event registration and attendance management.
- Club membership and participation tracking.
- Advanced content moderation and reporting.
- Campus analytics for administrators.
- Recommendation systems for events and campus content.
- Improved search and campus-specific personalization.
- Stronger security and production-scale monitoring.
- Google Play Store deployment and wider university rollout.

All items in this section are future scope and are not represented as currently implemented features.

12.4 Conclusion

Viragram demonstrates the development of a university-focused Android application that combines social networking with campus discovery. The project applies Java and object-oriented programming to a practical application containing model classes, activities, fragments, adapters and service managers. Firebase Authentication and Firestore provide cloud identity and data services, while ImageKit, Glide and Node.js/Express extend the media and backend capabilities.

The project also demonstrates practical software engineering work beyond simply creating screens. Requirements were considered, alternative ideas were evaluated, the application was developed in phases, integration problems were investigated, database permissions were corrected, Android configuration issues were fixed, image URL propagation was repaired and major workflows were tested again.

The result is a strong academic foundation for a campus-specific digital community platform. With future additions such as real-time chat, push notifications, event registration, moderation and analytics, the same architecture can be extended toward a more complete university ecosystem.

REFERENCES

Android Developers — Android developer documentation: https://developer.android.com/

Java Documentation — Oracle Java documentation: https://docs.oracle.com/en/java/

Firebase Documentation: https://firebase.google.com/docs

Firebase Authentication Documentation: https://firebase.google.com/docs/auth

Cloud Firestore Documentation: https://firebase.google.com/docs/firestore

ImageKit Documentation: https://imagekit.io/docs/

Node.js Documentation: https://nodejs.org/docs/

Express Documentation: https://expressjs.com/

Render Documentation: https://render.com/docs

GitHub Documentation: https://docs.github.com/

Glide Documentation: https://bumptech.github.io/glide/

Project-specific sources used for this report: the Viragram project presentation, supplied application screenshots, and the supplied project-report requirements/materials.

APPENDIX A — TEST CASE SUMMARY

ID	Area	Result
TC01–TC05	Registration, verification, login, recovery	Major authentication workflow tested
TC06–TC09	Posts, images, likes, comments	Tested; Firestore/image issues corrected
TC10–TC12	Search and follow	Tested
TC13–TC16	Stories, notifications, profile, admin access	Tested
TC17–TC20	Events, announcements, clubs, College Hub	Tested
TC21–TC22	Event and post detail images	Tested after URL-passing fixes
TC23–TC24	Dark mode and logout	Tested

APPENDIX B — IMPORTANT PROJECT STRUCTURE

Category	Components
Activities	LoginActivity, RegisterActivity, VerifyEmailActivity, ForgotPasswordActivity, MainActivity
Fragments	HomeFragment, SearchFragment, CreatePostFragment, NotificationsFragment, ProfileFragment
Adapters	StoryAdapter, PostAdapter, CommentAdapter
Models	User, Post, Story, Comment, Club, Event, NotificationItem, Conversation, ChatMessage
Utilities / Managers	Constants, UiUtils, LocalFeedManager, ViragramNotificationManager, LocalMessageManager
Services	Firebase Authentication, Cloud Firestore, Node.js / Express, ImageKit
Deployment	GitHub source management, Render backend deployment

APPENDIX C — FIVE-MINUTE FINAL DEMONSTRATION CHECKLIST

Step	Demonstration
1	Launch Viragram and show the login screen.
2	Show registration / college-email verification flow if credentials are available.
3	Open the Home Feed and explain stories and post interaction.
4	Create a post and demonstrate image selection/upload.
5	Show like and comment interaction.
6	Open Search and a student Profile.
7	Show Notifications and Messages.
8	Open College Hub and show Viragram Tech Fest 2026.
9	Demonstrate the admin portal and event/club/announcement management.
10	Explain Firebase, ImageKit and the Node/Express backend architecture.
11	Mention the main bugs that were solved.
12	End with the final result and future scope.